

# Домашнее задание № 13

1. Разворачиваем хранилище S3 с использованием MinIO
2. Устанавливаем clickhouse-backup и настраиваем политику хранения (storage policy) в конфигурации ClickHouse

 docker-compose.yml – Блокнот

Файл Правка Формат Вид Справка

```
services:
  minio:
    image: quay.io/minio/minio:latest
    container_name: minio
    ports:
      - "9000:9000" # S3 API
      - "9001:9001" # Web Console
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - minio-data:/data # ДОБАВЛЕНО: чтобы данные MinIO сохранялись при перезапуске
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - clickhouse-network

# Контейнер для автоматического создания бакетов при старте
init-minio:
  image: quay.io/minio/mc:latest
  container_name: init-minio
  depends_on:
    minio:
      condition: service_healthy
  entrypoint: >
    /bin/sh -c "
    /usr/bin/mc alias set myminio http://minio:9000 minioadmin minioadmin;
    /usr/bin/mc mb myminio/ch-native-s3;
    /usr/bin/mc mb myminio/ch-backups;
    exit 0;
    "
  networks:
    - clickhouse-network
```

```
clickhouse:
  image: clickhouse/clickhouse-server:latest
  container_name: clickhouse-server
  hostname: clickhouse-server
  restart: unless-stopped

  ports:
    - "${CLICKHOUSE_HTTP_PORT:-8123}:8123"
    - "${CLICKHOUSE_TCP_PORT:-9002}:9000"
    - "${CLICKHOUSE_INTERSERVER_PORT:-9009}:9009"

  environment:
    - CLICKHOUSE_DB=${CLICKHOUSE_DB:-default}
    - CLICKHOUSE_USER=${CLICKHOUSE_USER:-default}
    - CLICKHOUSE_PASSWORD=${CLICKHOUSE_PASSWORD}
    - CLICKHOUSE_DEFAULT_ACCESS_MANAGEMENT=1
    - CLICKHOUSE_LOG_LEVEL=information

  volumes:
    - ./config/listen.xml:/etc/clickhouse-server/config.d/listen.xml
    - ./config/optimized_config.xml:/etc/clickhouse-server/config.d/optimized_config.xml
    - ./config/storage_policy.xml:/etc/clickhouse-server/config.d/storage_policy.xml
    - clickhouse-data:/var/lib/clickhouse
    - clickhouse-logs:/var/log/clickhouse-server

  depends_on:
    init-minio:
      condition: service_completed_successfully

  ulimits:
    nofile:
      soft: 262144
      hard: 262144

  networks:
    - clickhouse-network

clickhouse-backup:
  image: altinity/clickhouse-backup:latest
  container_name: clickhouse-backup
  depends_on:
    - clickhouse
    - minio
  environment:
    - CLICKHOUSE_HOST=clickhouse-server
    - CLICKHOUSE_PORT=9000
    - CLICKHOUSE_USERNAME=${CLICKHOUSE_USER:-default}
    - CLICKHOUSE_PASSWORD=${CLICKHOUSE_PASSWORD}
  volumes:
    - ./clickhouse-backup/config.yml:/etc/clickhouse-backup/config.yml
    - clickhouse-data:/var/lib/clickhouse |
  networks:
    - clickhouse-network
  command: ["sleep", "infinity"]

volumes:
  minio-data:
    driver: local
  clickhouse-data:
    driver: local
  clickhouse-logs:
    driver: local
```

---

```

<clickhouse>
  <storage_configuration>
    <disks>
      <s3_disk>
        <type>s3</type>
        <endpoint>http://minio:9000/ch-native-s3/</endpoint>
        <access_key_id>minioadmin</access_key_id>
        <secret_access_key>minioadmin</secret_access_key>
        <region>us-east-1</region>
        <!-- Для MinIO часто требуется отключить проверку контрольных сумм -->
        <check_objects_after_upload>>false</check_objects_after_upload>
      </s3_disk>
    </disks>
    <policies>
      <s3_policy>
        <volumes>
          <main>
            <disk>s3_disk</disk>
          </main>
        </volumes>
      </s3_policy>
    </policies>
  </storage_configuration>
</clickhouse>

```

```

general:
  remote_storage: s3
  disable_progress_bar: false
  backups_to_keep_local: 2
  backups_to_keep_remote: 2

clickhouse:
  host: clickhouse-server
  port: 9000
  username: default
  password: ""

s3:
  access_key: minioadmin
  secret_key: minioadmin
  bucket: ch-backups
  endpoint: http://minio:9000
  region: us-east-1
  force_path_style: true
  disable_ssl: true
  compression_level: 1
  compression_format: tar
  path: "backups" # Папка внутри бакета для обычных метаданных и архивов
  object_disk_path: "object_disks" # Папка внутри бакета для данных S3-дисков (должна от

```

### 3. Создаем тестовую БД

```
-- 1. Создаем базу данных
CREATE DATABASE IF NOT EXISTS test_db;
USE test_db;

-- 2. Создаем таблицу users с ПРИНУДИТЕЛЬНЫМ использованием S3-политики (Задача 2 + 3)
CREATE TABLE IF NOT EXISTS users (
  id UInt32,
  name String,
  email String,
  created_at DateTime DEFAULT now()
) ENGINE = MergeTree()
ORDER BY id
SETTINGS storage_policy = 's3_policy';

-- 3. Создаем вторую таблицу (будет использовать диск по умолчанию)
CREATE TABLE IF NOT EXISTS orders (
  order_id UInt32,
  user_id UInt32,
  amount Float32,
  status String
) ENGINE = MergeTree()
ORDER BY order_id;

-- 4. Заполняем таблицы данными
INSERT INTO users (id, name, email) VALUES
(1, 'Alice', 'alice@test.com'),
(2, 'Bob', 'bob@test.com'),
(3, 'Charlie', 'charlie@test.com');

INSERT INTO orders (order_id, user_id, amount, status) VALUES
(101, 1, 150.50, 'completed'),
(102, 2, 89.99, 'pending'),
(103, 1, 200.00, 'completed');

-- 5. Проверяем, что данные на месте
SELECT '--- USERS ---' AS info;
SELECT * FROM users;
```

users 1 X

SELECT \* FROM users Введите SQL выражение чтобы отфильтровать результаты

|   | 123 id | AZ name | AZ email         | created_at          |
|---|--------|---------|------------------|---------------------|
| 1 | 1      | Alice   | alice@test.com   | 2026-09-15 00:33:37 |
| 2 | 2      | Bob     | bob@test.com     | 2026-09-15 00:33:37 |
| 3 | 3      | Charlie | charlie@test.com | 2026-09-15 00:33:37 |

```
SELECT '--- ORDERS ---' AS info;
SELECT * FROM orders;
```

orders 1 X

SELECT \* FROM orders Введите SQL выражение чтобы отфильтровать результаты

|   | 123 order_id | 123 user_id | 123 amount | AZ status |
|---|--------------|-------------|------------|-----------|
| 1 | 101          | 1           | 150,5      | completed |
| 2 | 102          | 2           | 89,99      | pending   |
| 3 | 103          | 1           | 200        | completed |

## Проверка, что данные ушли на S3

```
-- проверяем, что данные ушли в S3
SELECT
    table,
    name AS part_name,
    disk_name,
    bytes_on_disk
FROM system.parts
WHERE table IN ('users', 'orders') AND active
ORDER BY table;
```

parts 1 X

SELECT table, name AS part\_name, disk\_name, bytes\_on\_disk, ... Введите SQL выражение чтобы отфильтровать результаты

|   | AZ table | AZ part_name | AZ disk_name | 123 bytes_on_disk |
|---|----------|--------------|--------------|-------------------|
| 1 | orders   | all_1_1_0    | default      | 686               |
| 2 | users    | all_1_1_0    | s3_disk      | 717               |

## 4. Резервное копирование

- создаем бэкап локально внутри контейнера clickhouse-backup

```
$ winpty docker exec -it clickhouse-backup clickhouse-backup create my_first_backup
2026-09-14 22:23:19.414 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://clickhouse-server:9000
```

- загрузка этого бэкапа в наш MinIO

```
$ winpty docker exec -it clickhouse-backup clickhouse-backup upload my_first_backup
2026-09-14 22:27:44.018 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://clickhouse-server:9000
2026-09-14 22:27:44.018 INF pkg/clickhouse/clickhouse.go:1272 > SELECT value FROM 'system.' 'build_options' where name='VERSION_INTEGER'
```

- посмотреть локальные бэкапы

```
$ winpty docker exec -it clickhouse-backup clickhouse-backup list local
2026-09-14 22:31:37.077 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://clickhouse-server:9000
2026-09-14 22:31:37.077 INF pkg/clickhouse/clickhouse.go:1272 > SELECT value FROM 'system.' 'build_options' where name='VERSION_INTEGER'
2026-09-14 22:31:37.079 INF pkg/clickhouse/clickhouse.go:1272 > SELECT countIf(name='type') AS is_disk_type_present, countIf(name='object_storage_type') AS is_object_storage_type_present, countIf(name='free_space') AS is_free_space_present, countIf(name='disks') AS is_storage_policy_present, countIf(name='cache_path') AS is_cache_path_present FROM system.columns WHERE database='system' AND table IN ('disks', 'storage_policies')
2026-09-14 22:31:37.082 INF pkg/clickhouse/clickhouse.go:1272 > SELECT d.path AS path, argMin(d.name, d.cache_path != '') AS name, any(lower(if(d.type='ObjectStorage', d.object_storage_type, d.type))) AS type, min(d.free_space) AS free_space, groupUniqArray(s.policy_name) AS storage_policies FROM system.disks AS d LEFT JOIN (SELECT policy_name, arrayJoin(disks) AS disk FROM system.storage_policies) AS s ON s.disk = d.name GROUP BY d.path
2026-09-14 22:31:37.086 INF pkg/clickhouse/clickhouse.go:384 > clickhouse connection closed
my_first_backup 2026-09-14 22:23:19 local all:4.12KiB,data:1.70KiB,arch:0B,obj:1.29KiB,meta:1.14KiB,rbac:0B,conf:0B,nc:0B regular
```

- посмотреть бэкапы в удаленном хранилище (minio)

```
$ winpty docker exec -it clickhouse-backup clickhouse-backup list remote
2026-09-14 22:32:43.224 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://clickhouse-server:9000
2026-09-14 22:32:43.227 INF pkg/storage/general.go:307 > , list_duration=1.445478
2026-09-14 22:32:43.228 INF pkg/clickhouse/clickhouse.go:384 > clickhouse connection closed
my_first_backup 2026-09-14 22:27:44 remote all:22.54KiB,data:1.70KiB,arch:20.00KiB,obj:1.29KiB,meta:1.26KiB,rbac:0B,conf:0B,nc:0B regular
```

## 5. Повреждаем тестовые данные

- повреждаем тестовые данные
- 1. Полностью удаляем таблицу orders (имитация accidental DROP)  
**DROP TABLE IF EXISTS test\_db.orders;**
- 2. Удаляем пользователя с id = 2 (имитация accidental DELETE)  
**ALTER TABLE test\_db.users DELETE WHERE id = 2;**
- 3. Портим данные оставшегося пользователя (имитация повреждения данных)  
**ALTER TABLE test\_db.users UPDATE name = 'HACKED\_NAME', email = 'hacked@evil.com' WHERE id = 1;**

4. Проверяем "ущерб"

```
SELECT '--- USERS (должны быть повреждены/удалены) ---' AS info;
SELECT * FROM test_db.users ORDER BY id;
```

users 1 X

SELECT \* FROM test\_db.users ORDER BY id | Введите SQL выражение чтобы отфильтровать результаты

|   | 123 id | A-Z name    | A-Z email        | created_at          |
|---|--------|-------------|------------------|---------------------|
| 1 | 1      | HACKED_NAME | hacked@evil.com  | 2026-09-15 00:33:37 |
| 2 | 3      | Charlie     | charlie@test.com | 2026-09-15 00:33:37 |

```
SELECT '--- ORDERS (таблицы не существует, будет ошибка) ---' AS info;
SELECT * FROM test_db.orders;
```

users 1 X

ELECT \* FROM test\_db.orders | Введите SQL выражение чтобы отфильтровать результаты

```
SQL Error [60] [22000]: Code: 60. DB::Exception: Unknown table
expression identifier 'test_db.orders' in scope SELECT * FROM
test_db.orders LIMIT 0, 200. (UNKNOWN_TABLE) (queryId=
1dd4e443-9fb2-45f3-9537-8190b9833119)
```

## 6. Восстанавливаем данные из резервной копии

```
$ winpty docker exec -it clickhouse-backup clickhouse-backup restore_remote --rm my_first_backup
2026-09-14 22:42:24.357 INF pkg/clickhouse/clickhouse.go:165 > clickhouse connection success: tcp://clickhouse-server:9000
```

## 7. Проверяем, что поврежденные данные успешно восстановлены

```
-- проверочные запросы после восстановления из бэкапа
-- 1. Проверяем, что таблица orders вернулась с данными
SELECT '--- ORDERS (должны быть 3 строки) ---' AS info;
SELECT * FROM test_db.orders ORDER BY order_id;
```

orders 1 X

SELECT \* FROM test\_db.orders ORDER BY order\_id | Введите SQL выражение чтобы отфи

|   | 123 order_id | 123 user_id | 123 amount | A-Z status |
|---|--------------|-------------|------------|------------|
| 1 | 101          | 1           | 150,5      | completed  |
| 2 | 102          | 2           | 89,99      | pending    |
| 3 | 103          | 1           | 200        | completed  |

-- 2. Проверяем, что таблица users вернулась в исходное (неповрежденное) состояние  
**SELECT** '--- USERS (должны быть Alice, Bob, Charlie) ---' **AS** info;  
**SELECT** \* **FROM** test\_db.users **ORDER BY** id;

users 1 X

SELECT \* FROM test\_db.users ORDER BY id Введите SQL выражение чтобы отфильтровать результаты

|   | 123 id | A-Z name | A-Z email        | created_at          |
|---|--------|----------|------------------|---------------------|
| 1 | 1      | Alice    | alice@test.com   | 2026-09-15 00:33:37 |
| 2 | 2      | Bob      | bob@test.com     | 2026-09-15 00:33:37 |
| 3 | 3      | Charlie  | charlie@test.com | 2026-09-15 00:33:37 |

-- 3. САМОЕ ГЛАВНОЕ: Проверяем, что данные users действительно лежат на S3-диске,  
-- а не на локальном диске по умолчанию.

**SELECT**  
  **table**,  
  name **AS** part\_name,  
  disk\_name,  
  formatReadableSize(bytes\_on\_disk) **AS** size  
**FROM** system.parts  
**WHERE** database = 'test\_db' **AND** table **IN** ('users', 'orders') **AND** active  
**ORDER BY** table;

parts 1 X

SELECT table, name AS part\_name, disk\_name, formatReadableSize(bytes\_on\_disk) AS size Введите SQL выражение чтобы отфильтровать результаты

|   | A-Z table | A-Z part_name | A-Z disk_name | A-Z size |
|---|-----------|---------------|---------------|----------|
| 1 | orders    | all_1_1_0     | default       | 686.00 B |
| 2 | users     | all_1_1_0     | s3_disk       | 717.00 B |